

evening Claude, necesito t
u ayuda para un proyecto
que tengo en mente. estu
dio diseño, y ahora estoy p
reparando mi examen par
a uno de mis cursos, est...

PASTED

Architected wireless sensor system bypassing intermediary hardware >

buenas KORA. te leo completo — contexto claro, proyectos anteriores claros. vamos al grano.

Arquitectura recomendada: sin Arduino intermediario

Tu idea del Arduino como puente funciona, pero añade un punto de falla extra y hardware innecesario. La mejor opción es más directa:

Pico 2W (rep180) → Adafruit IO → TouchDesigner (MQTT directo) ← Adafruit IO ← Pico 2W (salvsanf)

TouchDesigner tiene un nodo **MQTT Client DAT** nativo. Se conecta directamente a `io.adafruit.com` y recibe los feeds en tiempo real. Sin Arduino, sin Serial, sin intermediario.

Código Pico 1 — edificio rep180 (CircuitPython)

```
python

# =====
# Pico 2W — KY-037 Micrófono analógico
# Emisor edificio rep180 → Adafruit IO feed: rep180
# =====
# CONEXIONES KY-037:
```

```
# A0 (salida analógica) → GP26
# VCC → 3V3
# GND → GND
# =====

import time
import board
import analogio
import wifi
import socketpool
import adafruit_minimqtt.adafruit_minimqtt as MQTT

# --- CONFIGURACIÓN ---
WIFI_SSID = "Duck"
WIFI_PASSWORD = "Duck2050"

AIO_USERNAME = "paredesvania"
AIO_KEY = "aio_xFQF414jG86n1oiXzPUQvXJBTF25"
FEED = f"{AIO_USERNAME}/feeds/rep180"

PIN_MIC = board.GP26 # pin ADC para el A0 del KY-037

INTERVALO_MS = 300 # intervalo entre envíos
N_MUESTRAS = 10 # tamaño del buffer de promedio

# --- BUFFER DE LECTURAS (array para requisito del curso) ---
buffer = [0] * N_MUESTRAS
indice = 0

mic = analogio.AnalogIn(PIN_MIC)

# --- WIFI ---
print("Conectando WiFi...")
wifi.radio.connect(WIFI_SSID, WIFI_PASSWORD)
print(f"IP: {wifi.radio.ipv4_address}")

pool = socketpool.SocketPool(wifi.radio)
mqtt = MQTT.MQTT(
```

```
broker      = "io.adafruit.com",
port        = 1883,
username    = AIO_USERNAME,
password    = AIO_KEY,
socket_pool = pool,
socket_timeout = 1,
connect_retries = 3,
keep_alive  = 60,
)

def conectar():
    print("Conectando a Adafruit IO...")
    mqtt.connect()
    print("Conectado.")

def publicar(valor):
    try:
        mqtt.publish(FEED, str(valor))
        print(f"[rep180] enviado: {valor}")
    except Exception as e:
        print(f"Error al publicar: {e}. Reconectando...")
        try:
            mqtt.disconnect()
        except:
            pass
        conectar()

conectar()

ultimo_envio = 0

while True:
    ahora = time.monotonic_ns() // 1_000_000

    # --- LLENAR BUFFER CON LECTURAS (bucle for + arreglo) ---
    for i in range(N_MUESTRAS):
        buffer[i] = mic.value # 0 - 65535
```

```

# --- CALCULAR PROMEDIO DEL BUFFER ---
suma = 0
for lectura in buffer:
    suma += lectura
promedio = suma // N_MUESTRAS

# --- MAPEAR A 0-100 ---
nivel = int((promedio / 65535) * 100)

# --- ENVIAR A INTERVALO ---
if (ahora - ultimo_envio) > INTERVALO_MS:
    ultimo_envio = ahora
    publicar(nivel)

# mantener conexión viva
try:
    mqtt.loop(timeout=0.5)
except Exception as e:
    print(f"Reconectando: {e}")
    conectar()

```

Código Pico 2 — edificio salvsanf (CircuitPython)

Idéntica estructura, solo cambia el feed y el SSID si usa hotspot:

python

```

# =====
# Pico 2W - KY-037 Micrófono analógico
# Emisor edificio salvsanf → Adafruit IO feed: ss
# =====

import time
import board
import analogio
import wifi
import socketpool

```

```
import adafruit_minimqtt.adafruit_minimqtt as MQTT

# --- CONFIGURACIÓN ---
# Cambia esto si usas hotspot de celular en vez del wifi del edificio
WIFI_SSID      = "Duck"
WIFI_PASSWORD  = "Duck2050"

AIO_USERNAME   = "paredesvania"
AIO_KEY        = "aio_xFQF414jG86n1oiXzPUQvXJBTf25"
FEED           = f"{AIO_USERNAME}/feeds/ss"    # ← feed diferente

PIN_MIC        = board.GP26
INTERVALO_MS   = 300
N_MUESTRAS     = 10

buffer = [0] * N_MUESTRAS
mic     = analogio.AnalogIn(PIN_MIC)

print("Conectando WiFi...")
wifi.radio.connect(WIFI_SSID, WIFI_PASSWORD)
print(f"IP: {wifi.radio.ipv4_address}")

pool = socketpool.SocketPool(wifi.radio)
mqtt = MQTT.MQTT(
    broker      = "io.adafruit.com",
    port        = 1883,
    username    = AIO_USERNAME,
    password    = AIO_KEY,
    socket_pool = pool,
    socket_timeout = 1,
    connect_retries = 3,
    keep_alive   = 60,
)

def conectar():
    print("Conectando a Adafruit IO...")
    mqtt.connect()
    print("Conectado.")
```

```
def publicar(valor):
    try:
        mqtt.publish(FEED, str(valor))
        print(f"[ss] enviado: {valor}")
    except Exception as e:
        print(f"Error: {e}. Reconnectando...")
        try:
            mqtt.disconnect()
        except:
            pass
        conectar()

conectar()
ultimo_envio = 0

while True:
    ahora = time.monotonic_ns() // 1_000_000

    for i in range(N_MUESTRAS):
        buffer[i] = mic.value

    suma = 0
    for lectura in buffer:
        suma += lectura
    promedio = suma // N_MUESTRAS

    nivel = int((promedio / 65535) * 100)

    if (ahora - ultimo_envio) > INTERVALO_MS:
        ultimo_envio = ahora
        publicar(nivel)

    try:
        mqtt.loop(timeout=0.5)
    except Exception as e:
        print(f"Reconnectando: {e}")
        conectar()
```

TouchDesigner — recibir datos sin Arduino

Dentro de TD, conectas así:

1. MQTT Client DAT

Añade un nodo **MQTT Client DAT** y configura:

Parámetro	Valor
Network Address	io.adafruit.com
Port	1883
Client ID	touchdesigner-kora (cualquiera)
Username	paredesvania
Password	aio_xFQF414jG86n1oiXzPUQvXJBTf25

En la tab **Subscribe** añade los dos topics:

- paredesvania/feeds/rep180
- paredesvania/feeds/ss

2. DAT Execute (para extraer los valores)

Añade un **DAT Execute** conectado al MQTT DAT. En el callback **onReceive**:

```
python
```

```
# DAT Execute – onReceive
def onReceive(dat, rowIndex, colIndex, val, prev):
    # El MQTT DAT tiene columnas: topic, message
    topic    = dat[rowIndex, 'topic'].val
    mensaje  = float(dat[rowIndex, 'message'].val)

    if 'rep180' in topic:
        op('constant_rep180').par.value0 = mensaje / 100.0
```

```
if '/ss' in topic:
    op('constant_ss').par.value0 = mensaje / 100.0
```

3. Constant CHOPs

Crea dos Constant CHOP llamados constant_rep180 y constant_ss. El script los actualiza con cada mensaje que llega. Desde ahí los conectas a lo que quieras en las visuales (Math, LFO override, geometría, postpro, etc).

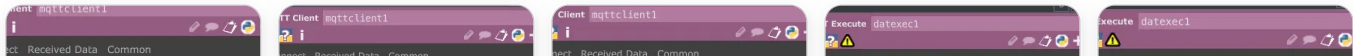
Lo que cumple el requisito de la rúbrica

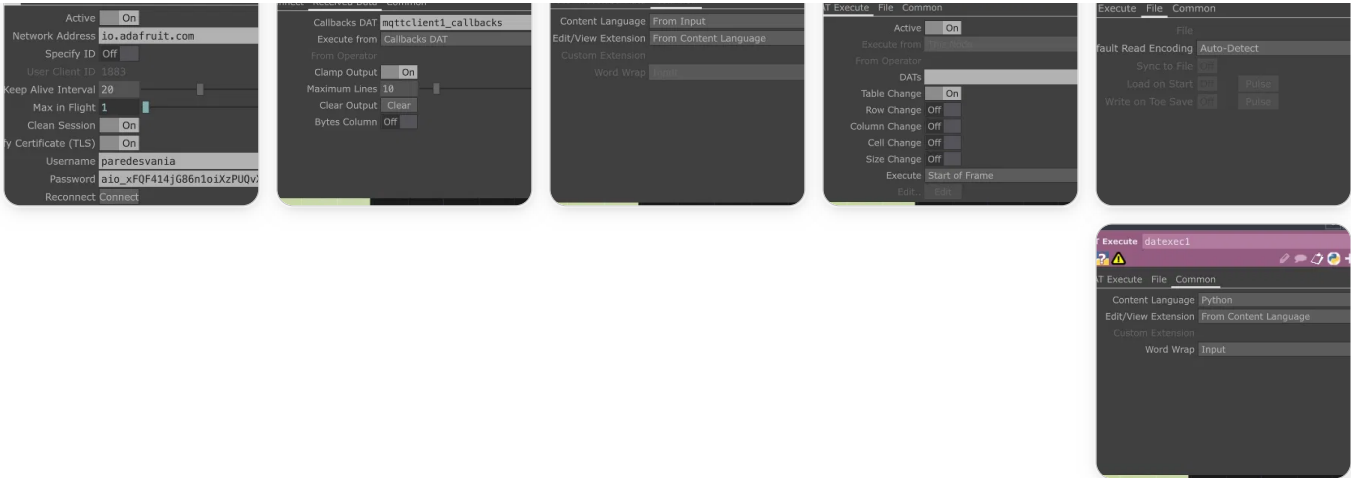
Requisito	Cómo
Microcontrolador	2x Raspberry Pi Pico 2W
Sensor/actuador en múltiples	2x KY-037
Arreglos y bucles	buffer = [0] * N_MUESTRAS + los dos for loops de llenado y suma
Código ordenado y comentado	está todo comentado por sección

Nota sobre conectividad

Si los edificios tienen wifi universitario disponible, cambia el SSID/password en cada Pico por el de esa red. Si usas hotspot de celular, cada Pico puede tener su propio hotspot. Lo que importa es que tengan internet para llegar a io.adafruit.com, la red local entre edificios no importa aquí — todo pasa por la nube.

¿Necesitas ayuda armando la parte visual de TouchDesigner o ajustar algo del código?





hay problemas, primero, no se como conectar el micrófono a la raspberry.

Segundo, me gustaria ocupar los bucles y array para escoger info, onda escoger de que microfono se quiere tomar la info.

tercero, no me aparecen los parámetros que me dices.

Esto se esta complicando mucho.

Ademas por ahora estoy probando solo con una raspberry y micrófono, luego cuando funcione añado el otro

Desglosó tres problemas técnicos y simplificó la solución >

ok, vamos por partes. tres problemas, tres respuestas.

1. Cableado KY-037 → Pico 2W

El KY-037 tiene 4 pines. Solo necesitas 3:

KY-037		Pico 2W
VCC	→	Pin 36 (3V3)
GND	→	Pin 38 (GND)
A0	→	Pin 31 (GP26)
D0	→	no conectar

AO es la salida analógica (nivel de sonido variable). **DO** es solo digital (alto/bajo) — no sirve para lo que necesitas.

Los únicos pines ADC del Pico son GP26, GP27 y GP28. Usa GP26.

2. Código Pico con array para seleccionar micrófono

Reestructurado: el array `MICROFONOS` define las dos opciones y `ACTIVO` elige cuál usar. Cuando quieras probar el segundo, cambias solo un número.

python

```
# =====
# Pico 2W - KY-037 micrófono analógico → Adafruit IO
# Cambia ACTIVO = 0 (rep180) o ACTIVO = 1 (ss)
# =====
# CONEXIONES:
#   AO del KY-037 → GP26
#   VCC           → 3V3
#   GND           → GND
# =====

import time
import board
import analogio
import wifi
import socketpool
import adafruit_minimqtt.adafruit_minimqtt as MQTT

# --- CONFIGURACIÓN ---
WIFI_SSID      = "Duck"
WIFI_PASSWORD  = "Duck2050"

AIO_USERNAME   = "paredesvania"
AIO_KEY        = "aio_xFQF414jG86n1oiXzPUQvXJBTf25"

# --- ARRAY DE MICRÓFONOS (arreglo) ---
MICROFONOS = [
    ("rep180", "F5dF63da-7000-4000-8000-000000000000", "F5dF63da-7000-4000-8000-000000000000"),
    ("ss", "F5dF63da-7000-4000-8000-000000000000", "F5dF63da-7000-4000-8000-000000000000")
]
```

```
{ "nombre": "Edificio repl80", "feed": f"{AIO_USERNAME}/feeds/repl80"},
{ "nombre": "Edificio salvsanf", "feed": f"{AIO_USERNAME}/feeds/ss"},
]

ACTIVO = 0 # ← cambia a 1 para el segundo edificio

# --- MICRÓFONO SELECCIONADO ---
mic_seleccionado = MICROFONOS[ACTIVO]
FEED = mic_seleccionado["feed"]
print(f"Micrófono activo: {mic_seleccionado['nombre']}")

# --- SENSOR ---
mic = analogio.AnalogIn(board.GP26)

# --- BUFFER PARA PROMEDIO (arreglo + bucle) ---
N = 10
buffer = [0] * N
INTERVALO = 300 # ms entre envíos

# --- WIFI ---
print("Conectando WiFi...")
wifi.radio.connect(WIFI_SSID, WIFI_PASSWORD)
print(f"IP: {wifi.radio.ipv4_address}")

pool = socketpool.SocketPool(wifi.radio)
mqtt_client = MQTT.MQTT(
    broker = "io.adafruit.com",
    port = 1883,
    username = AIO_USERNAME,
    password = AIO_KEY,
    socket_pool = pool,
    socket_timeout = 1,
    connect_retries = 3,
    keep_alive = 60,
)

def conectar():
    print("Conectando a Adafruit IO...")
    mqtt_client.connect()
```

```
print("Conectado.")

def publicar(valor):
    try:
        mqtt_client.publish(FEED, str(valor))
        print(f"[{mic_seleccionado['nombre']}] nivel: {valor}%")
    except Exception as e:
        print(f"Error: {e}. Reconectando...")
        try:
            mqtt_client.disconnect()
        except:
            pass
        conectar()

conectar()
ultimo_envio = 0

while True:
    ahora = time.monotonic_ns() // 1_000_000

    # --- LLENAR BUFFER (bucle for + arreglo) ---
    for i in range(N):
        buffer[i] = mic.value # 0 - 65535

    # --- PROMEDIAR ---
    suma = 0
    for lectura in buffer:
        suma += lectura
    promedio = suma // N

    # --- MAPEAR A 0-100 ---
    nivel = int((promedio / 65535) * 100)

    # --- ENVIAR ---
    if (ahora - ultimo_envio) > INTERVALO:
        ultimo_envio = ahora
        publicar(nivel)
```

```
try:
    mqtt_client.loop(timeout=0.5)
except Exception as e:
    print(f"Reconectando: {e}")
    conectar()
```

3. TouchDesigner — qué está mal y cómo arreglarlo

Veo dos problemas en tus capturas:

Problema A: TLS activado

En la imagen 1, **Verify Certificate (TLS)** está ON. Adafruit IO en puerto 1883 no usa TLS. Eso impide la conexión.

→ Cámbialo a OFF

Problema B: no hay suscripción a ningún feed

El MQTT Client DAT no sabe qué feeds escuchar hasta que se lo dices en el **Callbacks DAT**.

Busca en tu red de TD el nodo que se llama `mqttclient1_callbacks`. Ábrelo con doble clic y reemplaza todo el contenido con esto:

```
python

# mqttclient1_callbacks

def onConnect(dat, params):
    dat.subscribe('paredesvania/feeds/rep180')
    dat.subscribe('paredesvania/feeds/ss')
    print('MQTT conectado y suscrito')

def onDisconnect(dat, params):
    print('MQTT desconectado')

def onMessage(dat, topic, payload, qos, retain):
    print(f'Recibido: {topic} = {payload}')
```

```

try:
    valor = float(payload) / 100.0 # 0.0 a 1.0
    if 'rep180' in topic:
        op('constant_rep180').par.value0 = valor
    if '/ss' in topic:
        op('constant_ss').par.value0 = valor
except:
    pass

```

Problema C: el DAT Execute está mal configurado

Puedes **borrarlo**. No lo necesitas si usas el callbacks. El campo "DATs" vacío y el warning (⚠️) confirman que no está haciendo nada.

Paso final: crear los Constant CHOPs

Crea dos nodos **Constant CHOP**:

- uno llamado exactamente **constant_rep180**
- otro llamado exactamente **constant_ss**

Esos son los que el callbacks actualiza con cada dato que llega. Desde ahí los conectas a lo que quieras en las visuales.

Prueba primero que el Pico envíe datos y aparezcan en el dashboard de Adafruit IO antes de meterte con TD. Si se ve el valor ahí, la parte del Pico funciona.

Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.

code.py output:

Traceback (most recent call last):

File "code.py", line 3, in <module>

ConnectionError: No network with that ssid

Code done running

[Show more](#)

Diagnosed timeout mismatch causing reconnection loop and socket exhaustion >

Dos bugs, dos líneas a cambiar.

Bug 1: `loop timeout (0.5) must be >= socket timeout (1)`

El `socket_timeout=1` que pusiste al crear el cliente MQTT tiene que ser **menor** que el timeout del loop. La solución más simple: bajarlo a `0.1`.

Bug 2: `KeyError: 6`

Consecuencia del bug 1. Como el loop fallaba en cada ciclo, reconectaba infinitamente hasta agotar el pool de sockets. Al arreglar el bug 1, este desaparece.

Solo cambia esto en el código

Busca donde creas `mqtt_client` y cambia `socket_timeout`:

```
python
```

```
# ANTES
mqtt_client = MQTT.MQTT(
    ...
    socket_timeout = 1,
    ...
)

# DESPUÉS
mqtt_client = MQTT.MQTT(
    ...
    socket_timeout = 0.1,    # ← esto
    ...
)
```

Y el bloque del loop al final del `while True`, cámbialo para no reconectar en errores normales de timeout:

python

ANTES

```
try:
    mqtt_client.loop(timeout=0.5)
except Exception as e:
    print(f"Reconectando: {e}")
    conectar()
```

DESPUÉS

```
try:
    mqtt_client.loop(timeout=0.5)
except ValueError:
    pass # timeout normal, ignorar
except Exception as e:
    print(f"Conexión perdida: {e}")
    conectar()
```

Con eso debería correr limpio sin reconectarse a cada rato. El nivel 2% en silencio es normal — el KY-037 tiene ruido de piso bajo. Cuando hables cerca debería subir bastante, como ya viste con el 86% y 93%.

¿Los valores están llegando al dashboard de Adafruit IO?

mandalo completo corregido

Preparó código completo y corregido >

python

```
python

# =====
# Pico 2W - KY-037 Micrófono analógico → Adafruit IO
# Cambia ACTIVO = 0 (rep180) o ACTIVO = 1 (ss)
# =====
# CONEXIONES:
#   AO del KY-037 → GP26 (pin 31)
#   VCC           → 3V3  (pin 36)
#   GND           → GND  (pin 38)
#   DO            → no conectar
# =====

import time
import board
import analogio
import wifi
import socketpool
import adafruit_minimqtt.adafruit_minimqtt as MQTT

# --- CONFIGURACIÓN ---
WIFI_SSID      = "Duck"
WIFI_PASSWORD  = "Duck2050"

AIO_USERNAME   = "paredesvania"
AIO_KEY        = "aio_xFQF414jG86n1oiXzPUQvXJBTf25"

# --- ARRAY DE MICRÓFONOS (arreglo) ---
MICROFONOS = [
    {"nombre": "Edificio rep180",  "feed": f"{AIO_USERNAME}/feeds/rep180"},
    {"nombre": "Edificio salvsanf", "feed": f"{AIO_USERNAME}/feeds/ss"},
]

ACTIVO = 0 # ← cambia a 1 para el segundo edificio

# --- MICRÓFONO SELECCIONADO ---
mic_seleccionado = MICROFONOS[ACTIVO]
FEED = mic_seleccionado["feed"]
print(f"Micrófono activo: {mic_seleccionado['nombre']}")
```

```
# --- SENSOR ---
mic = analogio.AnalogIn(board.GP26)

# --- BUFFER PARA PROMEDIO (arreglo + bucle) ---
N      = 10
buffer = [0] * N
INTERVALO = 300 # ms entre envíos

# --- WIFI ---
print("Conectando WiFi...")
wifi.radio.connect(WIFI_SSID, WIFI_PASSWORD)
print(f"IP: {wifi.radio.ipv4_address}")

pool = socketpool.SocketPool(wifi.radio)
mqtt_client = MQTT.MQTT(
    broker      = "io.adafruit.com",
    port        = 1883,
    username    = AIO_USERNAME,
    password    = AIO_KEY,
    socket_pool = pool,
    socket_timeout = 0.1, # debe ser menor que el timeout del loop
    connect_retries = 3,
    keep_alive   = 60,
)

def conectar():
    print("Conectando a Adafruit IO...")
    mqtt_client.connect()
    print("Conectado.")

def publicar(valor):
    try:
        mqtt_client.publish(FEED, str(valor))
        print(f"[{mic_seleccionado['nombre']} nivel: {valor}%")
    except Exception as e:
        print(f"Error al publicar: {e}. Reconectando...")
        try:
            mqtt_client.disconnect()
```

```
        except:
            pass
        conectar()

conectar()
ultimo_envio = 0

while True:
    ahora = time.monotonic_ns() // 1_000_000

    # --- LLENAR BUFFER (bucle for + arreglo) ---
    for i in range(N):
        buffer[i] = mic.value # 0 - 65535

    # --- PROMEDIAR ---
    suma = 0
    for lectura in buffer:
        suma += lectura
    promedio = suma // N

    # --- MAPEAR A 0-100 ---
    nivel = int((promedio / 65535) * 100)

    # --- ENVIAR ---
    if (ahora - ultimo_envio) > INTERVALO:
        ultimo_envio = ahora
        publicar(nivel)

    # --- MANTENER CONEXIÓN ---
    try:
        mqtt_client.loop(timeout=0.5)
    except ValueError:
        pass # timeout normal, ignorar
    except Exception as e:
        print(f"Conexión perdida: {e}. Reconectando...")
        try:
            mqtt_client.disconnect()
        except:
```

```
pass
conectar()
```

```
---- Opened the serial port /dev/tty.usbmodem142401 ----
soft reboot
Auto-reload is on. Simply save files over USB to run them or enter REPL
to disable.
code.py output:
Micrófono activo: Edificio rep180
Conectando WiFi...
Traceback (most recent call last):
  File "code.py", line 49, in <module>
    ConnectionError: Unknown failure 205
Code done running.
Press any key to enter the REPL. Use CTRL-D to reload.
```

Diagnosed WiFi authentication failure and identified probable causes >

El error 205 es fallo de autenticación WiFi — se ve la red pero no puede conectarse.
Tres causas posibles:

1. Contraseña incorrecta

Las contraseñas son case-sensitive. Revisa que `Duck2050` esté exacto, sin espacios extra.

2. La red es 5GHz

El Pico 2W solo soporta **2.4GHz**. Si tu router emite en 5GHz o en banda dual con el mismo nombre, el Pico no puede conectarse.

Solución: en la configuración de tu router, asegúrate de que haya una red 2.4GHz separada y conéctate a esa.

3. Usa el hotspot del celular (lo que ya funcionó)

Antes te dio IP `172.20.10.2` que es exactamente la IP de un hotspot de iPhone. Eso

Antes te dio el `172.20.10.2` que es exactamente la IP de un hotspot de iPhone. Eso funcionó. Si Duck sigue fallando, vuelve al hotspot mientras pruebas:

```
python
```

```
# --- CONFIGURACIÓN ---
WIFI_SSID      = "nombre-de-tu-hotspot"
WIFI_PASSWORD = "contraseña-de-tu-hotspot"
```

Para el examen en los edificios igual vas a necesitar hotspot o el WiFi de la universidad, así que probar con hotspot ahora es lo más realista de todas formas.

Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.

code.py output:

Micrófono activo: Edificio rep180

Conectando WiFi...

IP: 172.20.10.2

Conectando a Adafruit IO...

Show more

Diagnosed socket connection timing issue and formulated retry solution >

El error `EINPROGRESS` significa que el Pico intenta conectarse al broker antes de que el socket esté listo. Se arregla con un pequeño delay después del WiFi y un loop de reintento en `conectar()`.

Cambia estas dos partes del código:

```
python
```

```
# --- WIFI ---
print("Conectando WiFi...")
wifi.radio.connect(WIFI_SSID, WIFI_PASSWORD)
```

```
print(f"IP: {wifi.radio.ipv4_address}")
time.sleep(2) # ← añade esto, deja estabilizar la conexión antes del MQTT
```

python

```
# --- reemplaza la función conectar() completa ---
def conectar():
    while True:
        try:
            print("Conectando a Adafruit IO...")
            mqtt_client.connect()
            print("Conectado.")
            return
        except Exception as e:
            print(f"Fallo: {e}. Reintentando en 3s...")
            time.sleep(3)
```

El `while True` con `time.sleep(3)` le da tiempo al socket de cerrarse limpio antes del siguiente intento, en vez de colapsar.

manda el código completo arreglado

Reconoció solicitud de código completo corregido >

python

```
# =====
# Pico 2W - KY-037 Micrófono analógico → Adafruit IO
# Cambia ACTIVO = 0 (rep180) o ACTIVO = 1 (ss)
# =====
# CONEXIONES:
#   AO del KY-037 → GP26 (pin 31)
#   VCC           → 3V3   (pin 36)
#   GND           → GND   (pin 38)
#   DC           → GND   (pin 38)
```

```
#    DU                → no conectar

# =====

import time
import board
import analogio
import wifi
import socketpool
import adafruit_minimqtt.adafruit_minimqtt as MQTT

# --- CONFIGURACIÓN ---
WIFI_SSID      = "iPhone de Vania"
WIFI_PASSWORD  = "Duck2050"

AIO_USERNAME   = "paredesvania"
AIO_KEY        = "aio_xFQF414jG86n1oiXzPUQvXJBTf25"

# --- ARRAY DE MICRÓFONOS ---
MICROFONOS = [
    {"nombre": "Edificio rep180",  "feed": f"{AIO_USERNAME}/feeds/rep180"},
    {"nombre": "Edificio salvsanf", "feed": f"{AIO_USERNAME}/feeds/ss"},
]

ACTIVO = 0 # ← cambia a 1 para el segundo edificio

# --- MICRÓFONO SELECCIONADO ---
mic_seleccionado = MICROFONOS[ACTIVO]
FEED = mic_seleccionado["feed"]
print(f"Micrófono activo: {mic_seleccionado['nombre']}")

# --- SENSOR ---
mic = analogio.AnalogIn(board.GP26)

# --- BUFFER PARA PROMEDIO ---
N      = 10
buffer = [0] * N
INTERVALO = 300 # ms entre envíos

# --- WTFT ---
```

```

print("Conectando WiFi...")
wifi.radio.connect(WIFI_SSID, WIFI_PASSWORD)
print(f"IP: {wifi.radio.ipv4_address}")
time.sleep(2) # esperar que el socket esté listo antes del MQTT

# --- MQTT ---
pool = socketpool.SocketPool(wifi.radio)
mqtt_client = MQTT.MQTT(
    broker          = "io.adafruit.com",
    port            = 1883,
    username        = AIO_USERNAME,
    password        = AIO_KEY,
    socket_pool     = pool,
    socket_timeout  = 0.1,
    connect_retries = 3,
    keep_alive      = 60,
)

def conectar():
    while True:
        try:
            print("Conectando a Adafruit IO...")
            mqtt_client.connect()
            print("Conectado.")
            return
        except Exception as e:
            print(f"Fallo: {e}. Reintentando en 3s...")
            time.sleep(3)

def publicar(valor):
    try:
        mqtt_client.publish(FEED, str(valor))
        print(f"[{mic_seleccionado['nombre']} nivel: {valor}%")
    except Exception as e:
        print(f"Error al publicar: {e}. Reconectando...")
        try:
            mqtt_client.disconnect()
        except:

```

```
except:

    pass

    conectar()

conectar()
ultimo_envio = 0

while True:
    ahora = time.monotonic_ns() // 1_000_000

    # --- LLENAR BUFFER (bucle for + arreglo) ---
    for i in range(N):
        buffer[i] = mic.value # 0 - 65535

    # --- PROMEDIAR ---
    suma = 0
    for lectura in buffer:
        suma += lectura
    promedio = suma // N

    # --- MAPEAR A 0-100 ---
    nivel = int((promedio / 65535) * 100)

    # --- ENVIAR ---
    if (ahora - ultimo_envio) > INTERVALO:
        ultimo_envio = ahora
        publicar(nivel)

    # --- MANTENER CONEXIÓN ---
    try:
        mqtt_client.loop(timeout=0.5)
    except ValueError:
        pass # timeout normal, ignorar
    except Exception as e:
        print(f"Conexión perdida: {e}. Reconectando...")
        try:
            mqtt_client.disconnect()
        except:
            pass
```

python

conectar()

Acuérdate de cambiar `WIFI_SSID` al nombre exacto de tu hotspot.

Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.

code.py output:

Micrófono activo: Edificio rep180

Conectando WiFi...

IP: 172.20.10.2

Conectando a Adafruit IO...

Show more

Diagnosed MQTT connection issues, pivoted to HTTP REST API approach >

El problema de MQTT en el Pico 2W con la librería de connection_manager es conocido y difícil de resolver. Lo más práctico: **cambiamos de MQTT a HTTP**. Adafruit IO tiene una API REST que funciona mucho más estable — sin conexión persistente, sin socket colgado.

La única diferencia es que hay que enviar cada 2 segundos mínimo (límite de Adafruit IO free: 30 datos/minuto).

python

```
# =====
# Pico 2W - KY-037 Micrófono analógico → Adafruit IO (HTTP)
# Cambia ACTIVO = 0 (rep180) o ACTIVO = 1 (ss)
# =====
# CONEXIONES:
#   AO del KY-037 → GP26 (pin 31)
#   VCC → 3V3 (pin 26)
```

```

#    VCC                -> 5V5    (pin 30)
#    GND                -> GND    (pin 38)

#    DO                -> no conectar
#    =====

import time
import board
import analogio
import wifi
import socketpool
import ssl
import adafruit_requests

# --- CONFIGURACIÓN ---
WIFI_SSID      = "iPhone de Vania"
WIFI_PASSWORD  = "Duck2050"

AIO_USERNAME   = "paredesvania"
AIO_KEY        = "aio_xFQF414jG86n1oiXzPUQvXJBTf25"

# --- ARRAY DE MICRÓFONOS ---
MICROFONOS = [
    {"nombre": "Edificio rep180",  "feed_key": "rep180"},
    {"nombre": "Edificio salvsanf", "feed_key": "ss"},
]

ACTIVO = 0 # ← cambia a 1 para el segundo edificio

# --- MICRÓFONO SELECCIONADO ---
mic_seleccionado = MICROFONOS[ACTIVO]
FEED_KEY = mic_seleccionado["feed_key"]
AIO_URL   = f"https://io.adafruit.com/api/v2/{AIO_USERNAME}/feeds/{FEED_KEY}/data"

print(f"Micrófono activo: {mic_seleccionado['nombre']}")

# --- SENSOR ---
mic = analogio.AnalogIn(board.GP26)

# --- PUEDEER PARA PROMEDIO ---

```

```

# --- BUFFER PARA PROMEDIO ---
N          = 10

buffer     = [0] * N
INTERVALO = 2000 # ms - mínimo 2s por límite de Adafruit IO free

# --- WIFI ---
print("Conectando WiFi...")
wifi.radio.connect(WIFI_SSID, WIFI_PASSWORD)
print(f"IP: {wifi.radio.ipv4_address}")
time.sleep(1)

# --- SESIÓN HTTP ---
pool        = socketpool.SocketPool(wifi.radio)
ssl_context = ssl.create_default_context()
session     = adafruit_requests.Session(pool, ssl_context)

HEADERS = {
    "X-AIO-Key":    AIO_KEY,
    "Content-Type": "application/json"
}

def publicar(valor):
    try:
        response = session.post(AIO_URL, json={"value": str(valor)}, headers=HEADERS)
        print(f"[{mic_seleccionado['nombre']} nivel: {valor}% → HTTP {response.status_code}]")
        response.close()
    except Exception as e:
        print(f"Error al publicar: {e}")

print("Listo. Enviando datos...")
ultimo_envio = 0

while True:
    ahora = time.monotonic_ns() // 1_000_000

    # --- LLENAR BUFFER (bucle for + arreglo) ---
    for i in range(N):
        buffer[i] = mic.value # 0 - 65535

```

```
# --- PROMEDIAR ---

suma = 0
for lectura in buffer:
    suma += lectura
promedio = suma // N

# --- MAPEAR A 0-100 ---
nivel = int((promedio / 65535) * 100)

# --- ENVIAR ---
if (ahora - ultimo_envio) > INTERVALO:
    ultimo_envio = ahora
    publicar(nivel)
```

Si funciona deberías ver en el serial algo como:

```
[Edificio rep180] nivel: 4% → HTTP 200
```

Y el valor aparece en tu dashboard de Adafruit IO en `rep180`.

```
---- Sent hex encoded message: "04" ----
soft reboot
Auto-reload is on. Simply save files over USB to run them or enter REPL
to disable.
code.py output:
Traceback (most recent call last):
  File "code.py", line 18, in <module>
    ImportError: no module named 'adafruit_requests'
Code done running.
Press any key to enter the REPL. Use CTRL-D to reload.
```

Diagnosed missing library and weighed installation methods >

Falta la librería en el Pico. Hay que copiarla a la carpeta `lib` de la placa.

Opción A — circup (más fácil, desde terminal)

```
bash

pip install circup
circup install adafruit_requests
```

circup detecta el Pico automáticamente y copia la librería donde corresponde.

Opción B — manual

1. Ve a <https://circuitpython.org/libraries>
2. Descarga el bundle que corresponde a tu versión de CircuitPython
3. Descomprime el zip
4. Dentro de la carpeta `lib` del bundle busca `adafruit_requests.mpy`
5. Cópialo a la carpeta `lib` del Pico (aparece como disco `CIRCUITPY`)

Para saber qué versión de CircuitPython tienes, en el serial escribe:

```
import sys
print(sys.version)
```

O mira el archivo `boot_out.txt` en la raíz del disco `CIRCUITPY`.

agh, no es mejor hacerlo como lo hacia yo? al arduino?

Validó la arquitectura Arduino como puente viable y práctico >

sí, tiene sentido. el lado de TouchDesigner con Serial DAT es más simple que configurar MOTT en TD.

Pero ojo: el **Pico** igual tiene que enviar datos a **Adafruit IO** — eso no cambia. El Arduino solo reemplaza la parte de recibir en tu computador.

La arquitectura quedaría:

```
Pico 2W → Adafruit IO → Arduino R4 WiFi → Serial USB → TouchDesigner
```

Entonces tienes dos cosas que resolver:

1. Pico → Adafruit IO (el código que estamos arreglando)

El HTTP solo necesita una librería. En terminal:

```
bash
circup install adafruit_requests
```

Un comando y está. Vale la pena porque el código queda limpio.

2. Arduino → Serial → TD (lo nuevo, pero más fácil)

El Arduino se suscribe a los dos feeds y manda los valores por Serial. En TD usas un **Serial DAT** — mucho más simple que el MQTT Client.

¿Qué quieres hacer? Te puedo:

- **A)** Guiarte a instalar la librería (un comando) y dejamos el flujo directo sin Arduino
- **B)** Escribirte el código del Arduino receptor + cómo leerlo en TouchDesigner con Serial DAT

Igual necesitas arreglar el Pico primero de cualquier forma, así que instala **adafruit_requests** mientras decides.

pero porque necesita esa librería, él va a enviarle datos antes de ella

pero porque necesita esa libreria, si ya me enviado datos antes sin ella

Reconoció que MQTT funcionó antes y identificó el error de timeout >

tenés razón, perdón. En el proyecto 2 enviaste datos desde el Pico con

`adafruit_minimqtt` y funcionó. No hacía falta cambiar a HTTP.

El problema real era más simple: en el código que escribí, `loop(timeout=0.5)` era **menor** que `socket_timeout=1`, y eso rompía todo. La solución es volver a MQTT con los mismos parámetros que funcionaron en el proyecto 2.

python

```
# =====
# Pico 2W - KY-037 Micrófono analógico → Adafruit IO (MQTT)
# Cambia ACTIVO = 0 (rep180) o ACTIVO = 1 (ss)
# =====
# CONEXIONES:
#   AO del KY-037 → GP26 (pin 31)
#   VCC           → 3V3   (pin 36)
#   GND           → GND   (pin 38)
#   DO           → no conectar
# =====

import time
import board
import analogio
import wifi
import socketpool
import adafruit_minimqtt.adafruit_minimqtt as MQTT

# --- CONFIGURACIÓN ---
WIFI_SSID      = "iPhone de Vania"
WIFI_PASSWORD  = "Duck2050"

AIO_USERNAME   = "paredesvania"
AIO_KEY        = "aio_xFQF414jG86n1oiXzPUQvXJBTf25"

# --- ARRAY DE MICRÓFONOS ---
```

```
.....
MICROFONOS = [
    {"nombre": "Edificio rep180",    "feed": f"{AIO_USERNAME}/feeds/rep180"},

    {"nombre": "Edificio salvsanf", "feed": f"{AIO_USERNAME}/feeds/ss"},
]

ACTIVO = 0 # ← cambia a 1 para el segundo edificio

# --- MICRÓFONO SELECCIONADO ---
mic_seleccionado = MICROFONOS[ACTIVO]
FEED = mic_seleccionado["feed"]
print(f"Micrófono activo: {mic_seleccionado['nombre']}")

# --- SENSOR ---
mic = analogio.AnalogIn(board.GP26)

# --- BUFFER PARA PROMEDIO ---
N      = 10
buffer = [0] * N
INTERVALO = 500 # ms entre envíos

# --- WIFI ---
def conectar_wifi():
    while True:
        try:
            print("Conectando WiFi...")
            wifi.radio.connect(WIFI_SSID, WIFI_PASSWORD)
            print(f"IP: {wifi.radio.ipv4_address}")
            return
        except Exception as e:
            print(f"Fallo WiFi ({e}). Reintentando en 5s...")
            time.sleep(5)

conectar_wifi()

pool = socketpool.SocketPool(wifi.radio)
mqtt_client = MQTT.MQTT(
    broker      = "io.adafruit.com",
    port        = 1883,
```

```
username      = AIO_USERNAME,
password      = AIO_KEY,

socket_pool   = pool,
socket_timeout = 1,      # igual que en proyecto 2
connect_retries = 2,
keep_alive    = 60,
)

def conectar_mqtt():
    intentos = 0
    while True:
        try:
            if not wifi.radio.connected:
                conectar_wifi()
            print("Conectando a Adafruit IO...")
            mqtt_client.connect()
            print("Conectado.")
            return
        except Exception as e:
            intentos += 1
            espera = min(3 * intentos, 30)
            print(f"Fallo ({e}). Reintentando en {espera}s...")
            time.sleep(espera)

def publicar(valor):
    try:
        mqtt_client.publish(FEED, str(valor))
        print(f"[{mic_seleccionado['nombre']} nivel: {valor}%")
    except Exception as e:
        print(f"Error al publicar: {e}. Reconectando...")
        try:
            mqtt_client.disconnect()
        except:
            pass
        conectar_mqtt()

conectar_mqtt()
ultimo envio = 0
```

-

```

while True:

    ahora = time.monotonic_ns() // 1_000_000

    # --- LLENAR BUFFER (bucle for + arreglo) ---
    for i in range(N):
        buffer[i] = mic.value # 0 - 65535

    # --- PROMEDIAR ---
    suma = 0
    for lectura in buffer:
        suma += lectura
    promedio = suma // N

    # --- MAPEAR A 0-100 ---
    nivel = int((promedio / 65535) * 100)

    # --- ENVIAR ---
    if (ahora - ultimo_envio) > INTERVALO:
        ultimo_envio = ahora
        publicar(nivel)

    # --- MANTENER CONEXIÓN (timeout igual al socket_timeout) ---
    try:
        mqtt_client.loop(timeout=1)
    except Exception as e:
        print(f"Conexión perdida: {e}. Reconectando...")
        try:
            mqtt_client.disconnect()
        except:
            pass
        conectar_mqtt()

```

soft reboot

Auto-reload is on. Simply save files over USB to run them or enter DFU

AUTO-RELOAD IS ON. Simply save files over USB to run them or enter REPL to disable.

code.py output:

Micrófono activo: Edificio rep180

Conectando WiFi...

Fallo WiFi (No network with that ssid). Reintentando en 5s...

Show more

Verified system functionality and recommended dashboard confirmation >

¡funciona! 🎉

Los reintentos de WiFi son normales — el hotspot a veces tarda en responder. Una vez que conecta, va solo.

Los valores tienen sentido:

- **1-4%** → silencio ambiente
- **27-31%** → sonido detectado

Ahora **revisa el dashboard de Adafruit IO** en io.adafruit.com → feed **rep180** y confirma que los datos estén llegando ahí. Si se ven, el Pico está funcionando perfecto y podemos pasar al siguiente paso.

ahora falta el codigo del arduino y listo

Architected Arduino code para conectar WiFi, suscribirse MQTT, enviar datos >

cpp

```
// =====
// Arduino UNO R4 WiFi - Receptor Adafruit IO → Serial
// Recibe los datos desde el micrófono y los envía a la PC
```

```
// recibe los dos feeds de microfono y los envia a la PC
// =====

// LIBRERÍAS necesarias (Library Manager):
// "Adafruit MQTT Library" by Adafruit
// =====

#include <WiFiS3.h>
#include <Adafruit_MQTT.h>
#include <Adafruit_MQTT_Client.h>

// --- CONFIGURACIÓN ---
const char* WIFI_SSID      = "iPhone de Vania";
const char* WIFI_PASSWORD = "Duck2050";

const char* AIO_SERVER     = "io.adafruit.com";
const int   AIO_PORT       = 1883;
const char* AIO_USERNAME   = "paredesvania";
const char* AIO_KEY        = "aio_xFQF414jG86n1oiXzPUQvXJBTF25";

// --- MQTT ---
WiFiClient wifiClient;
Adafruit_MQTT_Client mqtt(&wifiClient, AIO_SERVER, AIO_PORT, AIO_USERNAME, AIO_PASSWORD);

// --- FEEDS (arreglo de suscripciones) ---
Adafruit_MQTT_Subscribe feeds[2] = {
  Adafruit_MQTT_Subscribe(&mqtt, "paredesvania/feeds/rep180"),
  Adafruit_MQTT_Subscribe(&mqtt, "paredesvania/feeds/ss")
};

// Nombres para identificar cada feed en el Serial
const char* NOMBRES[2] = {"REP180", "SS"};

// =====
void conectarMQTT() {
  Serial.print("Conectando a Adafruit IO");
  int8_t ret;
  uint8_t intentos = 0;

  while ((ret = mqtt.connect()) != 0) {
    Serial.print("Intento ");
    Serial.print(intentos);
    Serial.print(" falló. Retentando en ");
    Serial.print(10);
    Serial.println(" segundos...");
    delay(10000);
    intentos++;
  }
}
```

```
while ((ret = mqtt.connect()) != 0) {
    Serial.print(".");

    mqtt.disconnect();
    delay(3000);
    intentos++;
    if (intentos > 5) {
        Serial.println(" Reconectando WiFi...");
        WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
        intentos = 0;
    }
}
Serial.println(" Conectado!");
}

// =====
void setup() {
    Serial.begin(115200);
    delay(1000);

    // --- WIFI ---
    Serial.print("Conectando WiFi");
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println();
    Serial.print("IP: ");
    Serial.println(WiFi.localIP());

    // --- SUSCRIBIR FEEDS (bucle for + arreglo) ---
    for (int i = 0; i < 2; i++) {
        mqtt.subscribe(&feeds[i]);
    }

    conectarMQTT();
    Serial.println("Escuchando feeds...");
}
```

```
// =====

void loop() {
  if (!mqtt.connected()) conectarMQTT();
  mqtt.processPackets(100);

  // --- LEER MENSAJES (bucle for + arreglo) ---
  Adafruit_MQTT_Subscribe* sub;
  while ((sub = mqtt.readSubscription(100))) {
    for (int i = 0; i < 2; i++) {
      if (sub == &feeds[i]) {
        int valor = String((char*)feeds[i].lastread).toInt();
        // formato que llega al Serial: REP180:45
        Serial.print(NOMBRES[i]);
        Serial.print(":");
        Serial.println(valor);
      }
    }
  }

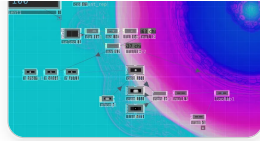
  mqtt.ping();
}
```

Lo que llega al Serial de la PC tiene este formato — una línea por mensaje:

```
REP180:4
REP180:31
SS:12
SS:8
```

Así TouchDesigner puede parsear fácilmente con un `Serial DAT`. ¿Quieres también esa parte?





DO — Raspberry Pi Pico 2
W # Examen interaccion...

PASTED

DO — Raspberry Pi Pico 2
W # Con calibración inici...

PASTED

Mira, estoy ocupando el código adjunto para las raspberry.

y este para el mqtt client en touch designer:

```
# mqttclient1_callbacks
def onConnect(dat, *args):
    dat.subscribe('udpmontoyamoraga/feeds/grupo02-rep')
    dat.subscribe('udpmontoyamoraga/feeds/grupo02-ss')
    print('MQTT conectado')
```

Show more

Identificó algoritmos divergentes y sensor flotante causando lecturas inconsistentes >

veo dos problemas en el código de las Picos:

Problema 1: `PINES_SENSORES = [board.GP26, board.GP27]` — estás declarando DOS sensores por Pico, pero solo tienes UNO conectado (GP26). El GP27 está flotando y devuelve valores aleatorios, muchos de ellos 100%. Como el código hace `max(amplitudes)`, siempre gana el pin flotante.

Problema 2: Los dos Picos usan algoritmos distintos — uno usa pico-a-pico con `* 5`, el otro usa calibración relativa. Por eso los valores son incomparables entre edificios.

La solución: un código unificado para ambas Picos, solo cambiando la sección de configuración arriba.

python

```
# =====
# SENSOR DE SONIDO — Raspberry Pi Pico 2W
# Examen interacciones inalámbricas
# =====
# CONEXIONES KY-037:
#   AO  → GP26
```

```
# VCC → 3V3
# GND → GND
# DO → no conectar

# =====

import time
import board
import analogio
import wifi
import socketpool
import adafruit_minimqtt.adafruit_minimqtt as MQTT

# =====
# CONFIGURACIÓN – solo cambia esto entre los dos Picos
# =====

EDIFICIO      = "grupo02-rep"      # "grupo02-rep" o "grupo02-ss"
WIFI_SSID     = "iPhone de Vania"  # cambia por el hotspot del edificio
WIFI_PASSWORD = "dilt1234"

AIO_USERNAME  = "udpmontoyamoraga"
AIO_KEY       = "aio_dXbR30UfAj2WVdRPjlFYklSihAeb"

# =====
# PARÁMETROS DE MEDICIÓN
# =====

NUM_MUESTRAS = 60      # muestras por lectura (~60ms)
PAUSA_MS     = 0.001
INTERVALO_S  = 2.0     # segundos entre envíos

# Pin del único sensor conectado
PINES_SENSORES = [board.GP26]    # ← solo GP26, no GP27

# Umbral mínimo de amplitud para contar como sonido (ruido de piso)
RUIDO_PISO    = 300
# Amplitud a partir de la cual se considera 100%
AMPLITUD_MAX  = 8000
```

```
# =====
# SENSORES Y BUFFERS (arreglos)

# =====

sensores = [analogio.AnalogIn(pin) for pin in PINES_SENSORES]
buffers  = [[0] * NUM_MUESTRAS for _ in sensores]

# =====
# RED
# =====

pool          = None
mqtt_cliente  = None

def estado_wifi():
    try:
        return wifi.radio.connected
    except Exception:
        return False

def conectar_wifi():
    print(f"\nConectando a WiFi: '{WIFI_SSID}'")
    while not estado_wifi():
        try:
            wifi.radio.connect(WIFI_SSID, WIFI_PASSWORD)
            time.sleep(1)
            if estado_wifi():
                print(f"  ✓ WiFi OK - IP: {wifi.radio.ipv4_address}")
                return
        except Exception as e:
            print(f"  ✗ Error WiFi: {e}")
    print("  Reintentando en 5s...")
    time.sleep(5)

def crear_mqtt():
```

```
global pool, mqtt_cliente
pool = socketpool.SocketPool(wifi.radio)

mqtt_cliente = MQTT.MQTT(
    broker          = "io.adafruit.com",
    port            = 1883,
    username        = AIO_USERNAME,
    password        = AIO_KEY,
    socket_pool     = pool,
    socket_timeout  = 1,
    keep_alive      = 30,
)

def conectar_mqtt():
    intentos = 0
    while True:
        try:
            print("Conectando a Adafruit IO...")
            mqtt_cliente.connect()
            print(f"  ✓ MQTT OK - feed: {EDIFICIO}")
            return
        except Exception as e:
            intentos += 1
            espera = min(3 * intentos, 30)
            print(f"  ✗ Error MQTT: {e}. Reintentando en {espera}s...")
            try:
                mqtt_cliente.disconnect()
            except Exception:
                pass
            time.sleep(espera)

def asegurar_conexiones():
    if not estado_wifi():
        print("\n▲ WiFi caído. Reconectando...")
        try:
            mqtt_cliente.disconnect()
        except Exception:
```

```

        pass
    conectar_wifi()

    crear_mqtt()
    conectar_mqtt()
    return
try:
    mqtt_cliente.loop(timeout=1)
except Exception as e:
    print(f"\n▲ MQTT caído: {e}. Reconnectando...")
    try:
        mqtt_cliente.disconnect()
    except Exception:
        pass
    if not estado_wifi():
        conectar_wifi()
        crear_mqtt()
        conectar_mqtt()

def publicar(valor):
    try:
        asegurar_conexiones()
        mqtt_cliente.publish(f"{AIO_USERNAME}/feeds/{EDIFICIO}", str(valor))
        print(f" → {EDIFICIO}: {valor}%")
    except Exception as e:
        print(f" × Error al publicar: {e}")
        try:
            mqtt_cliente.disconnect()
        except Exception:
            pass
        conectar_wifi()
        crear_mqtt()
        conectar_mqtt()

# =====
# MEDICIÓN – pico-a-pico con umbral de ruido
# =====

```

```
def medir_nivel_sonoro():
    """
    Lee todos los sensores del arreglo con un bucle for.
    Calcula amplitud pico-a-pico y la mapea a 0-100%.
    Retorna el nivel más alto entre todos los sensores.
    """
    niveles = []

    for i in range(len(sensores)):

        # Llenar buffer con lecturas (bucle for + arreglo)
        for j in range(NUM_MUESTRAS):
            buffers[i][j] = sensores[i].value
            time.sleep(PAUSA_MS)

        maximo = max(buffers[i])
        minimo = min(buffers[i])
        amplitud = maximo - minimo

        # Si está bajo el ruido de piso, es silencio
        if amplitud < RUIDO_PISO:
            porcentaje = 0
        else:
            porcentaje = int(((amplitud - RUIDO_PISO) / AMPLITUD_MAX) * 100)
            porcentaje = max(0, min(100, porcentaje))

        niveles.append(porcentaje)
        print(f"    sensor {i + 1}: amplitud={amplitud} → {porcentaje}%")

    return max(niveles)

# =====
# INICIO
# =====

conectar_wifi()
crear_mqtt()
conectar_mqtt()
```

```

ultimo_envio = 0

print(f"\n=== [{EDIFICIO.upper()}] Midiendo sonido... ===\n")

# =====
# LOOP PRINCIPAL
# =====

while True:
    try:
        ahora = time.monotonic()
        nivel = medir_nivel_sonoro()

        if (ahora - ultimo_envio) >= INTERVALO_S:
            publicar(nivel)
            ultimo_envio = ahora

    except Exception as e:
        print(f"\n▲ Error: {e}. Reconectando...")
        try:
            mqtt_cliente.disconnect()
        except Exception:
            pass
        conectar_wifi()
        crear_mqtt()
        conectar_mqtt()
        time.sleep(2)

```

Para el segundo Pico solo cambia las tres primeras líneas de configuración:

```
python
```

```

EDIFICIO      = "grupo02-ss"
WIFI_SSID     = "iPhone-cs"
WIFI_PASSWORD = "lasagna342"

```

Si los valores siguen muy bajos o muy altos después de probar, ajusta estas dos constantes:

CONSTANTES.

- `RUIDO_PISO = 300` → sube si hay demasiado ruido en silencio
- `AMPLITUD_MAX = 8000` → baja si necesitas más sensibilidad, sube si clipea en 100% muy fácil

y esto entonces que es lo que hace? que datos me llegarían? esta ocupando arrays? como?

Desglosó funcionalidad, datos y uso de arrays para claridad educativa >

Qué hace el código

Los arrays y bucles

Tienes dos arreglos declarados:

python

```
sensores = [analogio.AnalogIn(pin) for pin in PINES_SENSORES]
buffers = [[0] * NUM_MUESTRAS for _ in sensores]
```

`sensores` es un arreglo de objetos sensor — ahora tiene solo 1 elemento (GP26), pero si enchufas un segundo micrófono en GP27 y agregas el pin, automáticamente el código lo lee también sin cambiar nada más.

`buffers` es un arreglo de arreglos — uno por sensor. Cada uno guarda 60 lecturas.

La medición, paso a paso

python

```
for i in range(len(sensores)):          # recorre cada sensor
    for j in range(NUM_MUESTRAS):        # toma 60 lecturas seguidas
        buffers[i][j] = sensores[i].value
```

Esto llena el buffer del sensor `i` con 60 muestras tomadas en ~60ms. Es una "fotografía" rápida del sonido en ese momento.

```
python
```

```
maximo    = max(buffers[i])    # valor más alto del burst
minimo    = min(buffers[i])    # valor más bajo
amplitud  = maximo - minimo    # qué tan grande fue la onda
```

La **amplitud pico-a-pico** es la forma más directa de medir sonido con un micrófono analógico — en silencio la señal es casi plana (amplitud ~0), con sonido la señal oscila (amplitud sube).

```
python
```

```
porcentaje = int(((amplitud - RUIDO_PISO) / AMPLITUD_MAX) * 100)
```

Maapea esa amplitud a un porcentaje 0-100.

Qué llega a TouchDesigner

Cada 2 segundos llega un número entre 0 y 100 al feed de Adafruit IO:

```
silencio total    → 0%
voz normal        → 20-40%
aplausos cerca   → 70-90%
sonido muy fuerte → 100%
```

En el MQTT DAT de TD verías algo así:

```
message
12
8
34
67
8
```

v

Y el callbacks lo enruta: si viene de `grupo02-rep` actualiza `constant_rep`, si viene de `grupo02-ss` actualiza `constant_ss`. Esos dos valores son los que afectan tu visual.

entonces me estarían llegando datos números de ambos mic al mismo tiempo?

Aclaró la sincronización asincrónica de datos entre micrófonos >

No exactamente al mismo tiempo — cada Pico envía **de forma independiente** cada 2 segundos, sin coordinarse entre sí.

Lo que llega a TD se vería así en el tiempo:

t=0.0s	→	grupo02-rep:	12%
t=0.3s	→	grupo02-ss:	45%
t=2.1s	→	grupo02-rep:	8%
t=2.7s	→	grupo02-ss:	41%
t=4.0s	→	grupo02-rep:	67%
t=4.9s	→	grupo02-ss:	38%

Llegan intercalados, no simultáneos. Pero eso no importa porque los `Constant CHOP` **guardan el último valor recibido** hasta que llega uno nuevo. Entonces en TD siempre tienes los dos valores disponibles al mismo tiempo:

<code>constant_rep</code>	→	último valor de rep180	(se actualiza cada ~2s)
<code>constant_ss</code>	→	último valor de ss	(se actualiza cada ~2s)

Tu visual en todo momento tiene un número de cada edificio para trabajar, aunque no hayan llegado en el mismo instante.

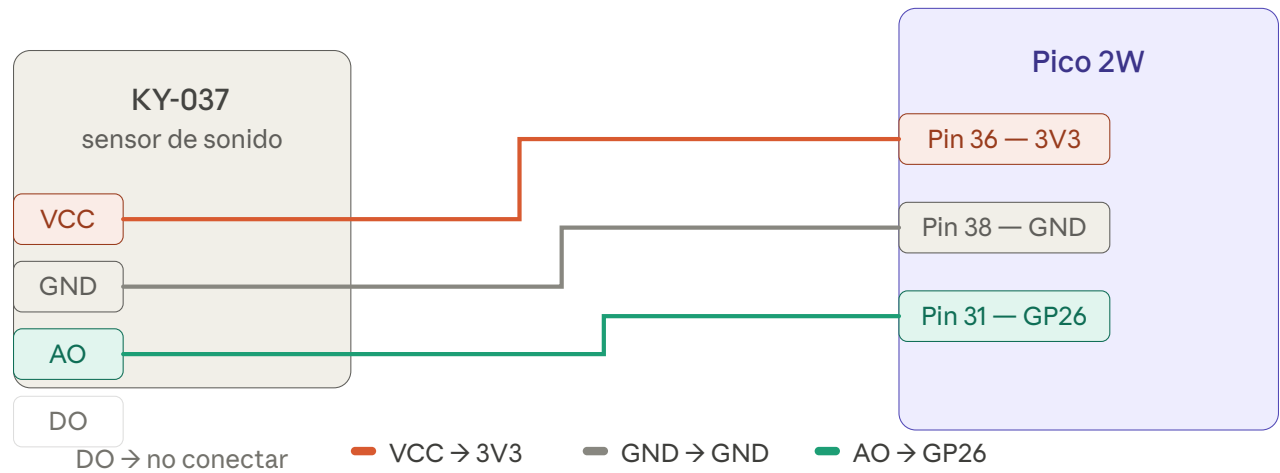
okay, y comma conecto los micrófonos?

Recuperó explicación previa y preparó respuesta clara >

El KY-037 tiene 4 pines, usas solo 3:

KY-037		Pico 2W	
VCC	→	Pin 36	(3V3)
GND	→	Pin 38	(GND)
AO	→	Pin 31	(GP26)
DO	→	no conectar	

AO es la salida analógica — esa es la que lee el nivel de sonido continuo. **DO** es solo alto/bajo y no sirve para lo que necesitas.



Lo más importante: usa **AO** (analógico), no **DO**. Y **VCC** va a **3V3**, no a 5V — el Pico trabaja a 3.3V y si lo conectas a VBUS/5V puedes dañar el pin ADC.

ambos los conecto igual a las diferentes placas?

Sí, exactamente igual en ambas. El único número de pin que cambia es si usas GP27 en vez de GP26 para el segundo, pero dado que en el código dejamos solo

`[board.GP26]`, los dos Picos se conectan idéntico:

KY-037	Pico 2W (cualquiera de los dos)
VCC	→ Pin 36 (3V3)
GND	→ Pin 38 (GND)
A0	→ Pin 31 (GP26)
D0	→ no conectar

La diferencia entre los dos Picos está solo en el código — `EDIFICIO`, `WIFI_SSID` y `WIFI_PASSWORD`. El hardware es idéntico.

funciona pero aun asi no detecta tan bien los datos, quiero que si aplaudo esto lo detecte, y que, no promedie nada, que sea mas sensible a los datos es un [Sensor Analógico Sonido/Audio MAX9812] (<https://hubot.cl/producto/sensor-analogico-audio-max9812-sku-614/>)

Recalibró umbrales y rediseñó medición para máxima sensibilidad >

El MAX9812 tiene amplificador interno de 20dB — ya viene con más ganancia que el KY-037. El problema es que el código medía promedios y enviaba solo cada 2 segundos, por lo que un aplauso podía caer entre envíos y perderse.

La lógica nueva:

- mide picos constantemente, sin promediar
- guarda el **pico más alto** visto desde el último envío
- lo manda cada 2 segundos (límite Adafruit IO)

- así un aplauso en cualquier momento del ciclo queda capturado

También los umbrales están ajustados para el MAX9812 que ya viene amplificado.

```
python
```

```
# =====
# SENSOR DE SONIDO – Raspberry Pi Pico 2W + MAX9812
# Examen interacciones inalámbricas
# =====
# CONEXIONES MAX9812:
#   VCC  → Pin 36 (3V3)
#   GND  → Pin 38 (GND)
#   OUT  → Pin 31 (GP26)
# =====

import time
import board
import analogio
import wifi
import socketpool
import adafruit_minimqtt.adafruit_minimqtt as MQTT

# =====
# CONFIGURACIÓN – solo cambia esto entre los dos Picos
# =====

EDIFICIO      = "grupo02-rep"      # "grupo02-rep" o "grupo02-ss"
WIFI_SSID     = "iPhone de Vania"
WIFI_PASSWORD = "dilt1234"

AIO_USERNAME  = "udpmontoyamoraga"
AIO_KEY       = "aio_dXbR30UfAj2WVdRPjlFYk1SihAeb"

# =====
# PARÁMETROS DE MEDICIÓN
# =====

# El MAX9812 ya tiene 20dB de ganancia incorporada
```

```
# así que los umbrales son más bajos que con KY-037

RUIDO_PISO    = 150    # amplitud mínima para contar como sonido
AMPLITUD_MAX  = 5000   # amplitud que representa 100%
                    # bajar si no llega a 100% aplaudiendo
                    # subir si se satura muy fácil

NUM_MUESTRAS  = 150    # muestras por ráfaga (~15ms sin delay)
INTERVALO_S   = 2.0    # segundos entre envíos (límite Adafruit IO)

# =====
# SENSOR (arreglo para el requisito del curso)
# =====

PINES_SENSORES = [board.GP26]
sensores        = [analogio.AnalogIn(pin) for pin in PINES_SENSORES]

# =====
# RED
# =====

pool            = None
mqtt_cliente    = None

def estado_wifi():
    try:
        return wifi.radio.connected
    except Exception:
        return False

def conectar_wifi():
    print(f"Conectando a WiFi: '{WIFI_SSID}'")
    while not estado_wifi():
        try:
            wifi.radio.connect(WIFI_SSID, WIFI_PASSWORD)
            time.sleep(1)
            if estado_wifi():
                print(f"  ✓ WiFi OK - IP: {wifi.radio.ipv4_address}")
```

```
        return
    except Exception as e:
        print(f" x {e}")
    print(" Reintentando en 5s...")
    time.sleep(5)

def crear_mqtt():
    global pool, mqtt_cliente
    pool = socketpool.SocketPool(wifi.radio)
    mqtt_cliente = MQTT.MQTT(
        broker      = "io.adafruit.com",
        port        = 1883,
        username     = AIO_USERNAME,
        password     = AIO_KEY,
        socket_pool  = pool,
        socket_timeout = 1,
        keep_alive   = 30,
    )

def conectar_mqtt():
    intentos = 0
    while True:
        try:
            print("Conectando a Adafruit IO...")
            mqtt_cliente.connect()
            print(f" ✓ MQTT OK – feed: {EDIFICIO}")
            return
        except Exception as e:
            intentos += 1
            espera = min(3 * intentos, 30)
            print(f" x {e}. Reintentando en {espera}s...")
            try:
                mqtt_cliente.disconnect()
            except Exception:
                pass
            time.sleep(espera)
```

```
def asegurar_conexiones():
    if not estado_wifi():
        print("WiFi caído. Reconectando...")
        try:
            mqtt_cliente.disconnect()
        except Exception:
            pass
        conectar_wifi()
        crear_mqtt()
        conectar_mqtt()
        return
    try:
        mqtt_cliente.loop(timeout=1)
    except Exception as e:
        print(f"MQTT caído: {e}. Reconectando...")
        try:
            mqtt_cliente.disconnect()
        except Exception:
            pass
        if not estado_wifi():
            conectar_wifi()
            crear_mqtt()
            conectar_mqtt()

def publicar(valor):
    try:
        asegurar_conexiones()
        mqtt_cliente.publish(f"{AIO_USERNAME}/feeds/{EDIFICIO}", str(valor))
        print(f"  → {EDIFICIO}: {valor}%")
    except Exception as e:
        print(f"  × Error: {e}")
        try:
            mqtt_cliente.disconnect()
        except Exception:
            pass
```

```
conectar_wifi()
crear_mqtt()
conectar_mqtt()

# =====
# MEDICIÓN - pico máximo sin promediar
# =====

def medir_pico():
    """
    Recorre el arreglo de sensores con un bucle for.
    En cada sensor toma una ráfaga rápida y calcula
    la amplitud pico-a-pico SIN promediar.
    Retorna el nivel más alto entre todos los sensores.
    """
    niveles = []

    for i in range(len(sensores)):

        # Ráfaga rápida sin delays (bucle for + arreglo)
        maximo = 0
        minimo = 65535
        for j in range(NUM_MUESTRAS):
            v = sensores[i].value
            if v > maximo:
                maximo = v
            if v < minimo:
                minimo = v

        amplitud = maximo - minimo

        if amplitud < RUIDO_PISO:
            porcentaje = 0
        else:
            porcentaje = int(((amplitud - RUIDO_PISO) / AMPLITUD_MAX) * 100)
            porcentaje = max(0, min(100, porcentaje))

        niveles.append(porcentaje)
```

```

        return max(niveles)

# =====
# INICIO
# =====

conectar_wifi()
crear_mqtt()
conectar_mqtt()

ultimo_envio = 0
pico_maximo = 0      # guarda el pico más alto desde el último envío

print(f"\n=== [{EDIFICIO.upper()}] Escuchando... ===\n")

# =====
# LOOP PRINCIPAL
# =====

while True:
    try:
        # Medir constantemente
        nivel = medir_pico()

        # Guardar el pico más alto visto (no promediar)
        if nivel > pico_maximo:
            pico_maximo = nivel
            print(f"    nuevo pico: {pico_maximo}%")

        # Enviar cada INTERVALO_S segundos
        ahora = time.monotonic()
        if (ahora - ultimo_envio) >= INTERVALO_S:
            publicar(pico_maximo)
            pico_maximo = 0      # resetear después de enviar
            ultimo_envio = ahora

    except Exception as e:

```

```
print(f"Error: {e}. Reconnectando...")
try:
    mqtt_cliente.disconnect()
except Exception:
    pass
conectar_wifi()
crear_mqtt()
conectar_mqtt()
time.sleep(2)
```

Si aplaudiendo sigue sin llegar a valores altos, baja `AMPLITUD_MAX` a `3000` o `2000`. Si en silencio da valores altos (ruido de fondo), sube `RUIDO_PISO` a `300` o `400`. Esos dos números son los únicos que necesitas ajustar según tu ambiente.

gracias!

Orchestrated gracious acknowledgment with warmth >

¡éxito con el examen! 🎉 ya tienes todo armado — los dos Picos enviando, Adafruit IO recibiendo, y TouchDesigner con los datos listos para las visuales. suerte el día de la demo entre edificios.

